

24.3 — Async/Await & Concurrency

Goal: `async/await` শুধু কীভাবে লিখতে হয় তা না—ভিতরে কীভাবে Promise-এর সাথে কাজ করে, কোথায় sequential execution দরকার, কোথায় concurrent execution দরকার, এবং কোন Promise combinator কখন ব্যবহার করতে হবে—এসব পরিষ্কারভাবে বোঝা।

1. `async` Function

JavaScript-এ কোনো function-এর আগে `async` লিখলে সেটি একটি **async function** হয়ে যায়।

```
async function greet() {  
  return "Hello";  
}
```

এখন:

```
const result = greet();  
  
console.log(result);
```

Output:

```
Promise { "Hello" }
```

Important Rule

An `async` function always returns a Promise.

এমনকি তুমি normal value return করলেও Promise পাওয়া যাবে।

```
async function getNumber() {  
  return 10;  
}  
  
getNumber().then((value) => {  
  console.log(value);  
});
```

Output:

```
10
```

Conceptually:

```
return 10;
```

behaves like:

```
return Promise.resolve(10);
```

2. `async` Function থেকে Object Return

```
async function getUser() {  
  return {  
    id: 1,  
    name: "Sharlar"  
  };  
}
```

সরাসরি object পাওয়া যাবে না:

```
const user = getUser();  
  
console.log(user.name);
```

এটা কাজ করবে না, কারণ `user` হলো Promise।

সঠিক:

```
const user = await getUser();  
  
console.log(user.name);
```

অথবা:

```
getUser().then((user) => {  
  console.log(user.name);  
});
```

3. `await` কী?

`await` একটি Promise-এর result পাওয়ার জন্য অপেক্ষা করার মতো syntax ।

```
const result = await promise;
```

Example:

```
async function getData() {  
  const result = await fetch("/api/users");  
  
  console.log(result);  
}
```

এখানে `await` Promise settle হওয়া পর্যন্ত ওই async function-এর execution pause করে।

4. `await` কি পুরো JavaScript Thread Block করে?

না।

এটা খুব গুরুত্বপূর্ণ।

```
async function test() {  
  console.log("A");  
  
  await somePromise;  
  
  console.log("B");  
}  
  
console.log("Start");  
  
test();
```

```
console.log("End");
```

`await` পুরো JavaScript execution বন্ধ করে না।

বরং:

```
async function
  ↓
await
  ↓
এই function-এর execution pause
  ↓
JavaScript অন্য কাজ করতে পারে
  ↓
Promise settle হলে
  ↓
function পরে resume করে
```

মনে রাখবে

`await` blocks the async function's continuation, **not the entire JavaScript thread**.

5. `await` কী Return করে?

```
const value = await promise;
```

Promise fulfill হলে:

```
const value
```

এর মধ্যে fulfilled value পাওয়া যায়।

Example:

```
const promise = Promise.resolve(100);

const value = await promise;
```

```
console.log(value);
```

Output:

```
100
```

6. `await` **Normal Value**-এর ক্ষেত্রেও কাজ করে

`await` শুধু Promise-এর জন্য নয়।

```
const value = await 10;  
  
console.log(value);
```

Output:

```
10
```

Conceptually:

```
await 10
```

roughly behaves like:

```
await Promise.resolve(10)
```

7. `await` **Thenable**-এর সাথেও কাজ করতে পারে

JavaScript Promise-এর মতো আচরণ করা object-কে **thenable** বলা হয়।

```
const thenable = {  
  then(resolve) {  
    resolve("Hello");  
  }  
};
```

```
}  
};
```

এখন:

```
const result = await thenable;  
  
console.log(result);
```

Output:

```
Hello
```

এখানে JavaScript object's `then()` method detect করে Promise-like behavior handle করে।

8. `async` + `await` Basic Example

```
function getUser() {  
  return Promise.resolve({  
    id: 1,  
    name: "Shariar"  
  });  
}  
  
async function main() {  
  const user = await getUser();  
  
  console.log(user);  
}  
  
main();
```

এটা Promise chain-এর equivalent:

```
function main() {  
  return getUser().then((user) => {  
    console.log(user);  
  });  
}
```

9. Promise Chaining বনাম Async/Await

Promise chaining

```
getUser()
  .then((user) => {
    return getPosts(user.id);
  })
  .then((posts) => {
    return getComments(posts[0].id);
  })
  .then((comments) => {
    console.log(comments);
  })
  .catch((error) => {
    console.error(error);
  });
```

Async/Await

```
async function main() {
  try {
    const user = await getUser();

    const posts = await getPosts(user.id);

    const comments = await getComments(posts[0].id);

    console.log(comments);
  } catch (error) {
    console.error(error);
  }
}
```

দুটোই valid ।

`async/await` অনেক সময় complex async flow পড়তে সহজ করে ।

10. `return` in Async Function

```
async function getValue() {  
  return 100;  
}
```

এটা:

```
Promise.resolve(100)
```

এর মতো behave করে।

তাই:

```
const value = await getValue();  
  
console.log(value);
```

Output:

```
100
```

11. `throw` in Async Function

```
async function test() {  
  throw new Error("Something went wrong");  
}
```

এটা synchronous exception হিসেবে বাইরে সরাসরি ছুড়ে দেয় না।

বরং returned Promise **reject** হয়।

```
test().catch((error) => {  
  console.log(error.message);  
});
```

Output:

Something went wrong

Conceptually:

```
async function test() {  
  throw new Error("Failed");  
}
```

behaves roughly like:

```
function test() {  
  return Promise.reject(new Error("Failed"));  
}
```

12. try/catch + await

Async code-এর error handling-এর সবচেয়ে common pattern:

```
async function getData() {  
  try {  
    const response = await fetch("/api/users");  
  
    const data = await response.json();  
  
    return data;  
  } catch (error) {  
    console.error("Failed:", error);  
  }  
}
```

যদি awaited Promise reject করে:

```
await promise
```

তাহলে `catch` block execute করবে।

13. try/catch কোথায় কাজ করে?

```
async function test() {  
  try {  
    const result = await Promise.reject(  
      new Error("Failed")  
    );  
  
    console.log(result);  
  } catch (error) {  
    console.log("Caught:", error.message);  
  }  
}  
  
test();
```

Output:

```
Caught: Failed
```

14. Floating Promise + try/catch

এখানে একটি খুব important mistake:

```
async function test() {  
  try {  
    doSomethingAsync();  
  } catch (error) {  
    console.log("Caught");  
  }  
}
```

যদি `doSomethingAsync()` Promise return করে এবং পরে reject করে, তাহলে এই `try/catch` সেটা ধরবে না।

কারণ Promise-টাকে `await` করা হয়নি।

সঠিক:

```
async function test() {  
  try {  
    await doSomethingAsync();  
  } catch (error) {  
    console.log("Caught");  
  }  
}
```

Rule

যদি async error-কে local `try/catch` দিয়ে ধরতে চাও, সাধারণত Promise-টাকে `await` করতে হবে।

15. `finally` with Async/Await

```
async function loadData() {  
  try {  
    const data = await fetchData();  
  
    return data;  
  } catch (error) {  
    console.error(error);  
  } finally {  
    console.log("Loading finished");  
  }  
}
```

Success হলেও:

Loading finished

Error হলেও:

Loading finished

16. Sequential Execution

ধরো তিনটি API call আছে:

```
const user = await getUser();

const posts = await getPosts();

const comments = await getComments();
```

এখানে:

```
getUser
  ↓
complete
  ↓
getPosts
  ↓
complete
  ↓
getComments
```

এটা **sequential execution** ।

17. Sequential Execution কখন দরকার?

যখন পরের কাজ আগের কাজের result-এর ওপর নির্ভর করে ।

Example:

```
const user = await getUser();

const orders = await getOrders(user.id);

const payment = await getPayment(orders[0].paymentId);
```

এখানে dependency আছে ।

```
user
  ↓
orders
  ↓
payment
```

তাই sequential execution natural ।

18. Sequential Execution-এর Performance Problem

ধরো:

```
API A = 2 sec  
API B = 3 sec  
API C = 1 sec
```

Sequential:

```
A → 2s  
B → 3s  
C → 1s  
  
Total ≈ 6s
```

কিন্তু যদি তিনটিই independent হয়, তাহলে এইভাবে করা unnecessary slow ।

19. Independent Async Operations

ধরো:

```
const users = await getUsers();  
const products = await getProducts();  
const categories = await getCategories();
```

যদি তিনটি operation একে অপরের ওপর depend না করে, তাহলে sequential execution করার দরকার নেই ।

Better:

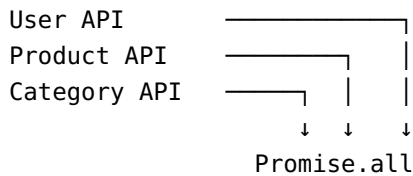
```
const usersPromise = getUsers();  
const productsPromise = getProducts();  
const categoriesPromise = getCategories();  
  
const [users, products, categories] = await Promise.all([  
  usersPromise,  
  productsPromise,
```

```
categoriesPromise  
]);
```

এখন operations overlap করতে পারে।

20. Concurrent Execution

Independent async operations একসাথে start করা হলে তারা **concurrently** এগোতে পারে।



ধরো:

```
users = 2s  
products = 3s  
categories = 1s
```

Sequential:

```
2 + 3 + 1 = 6s
```

Concurrent:

```
max(2, 3, 1) ≈ 3s
```

বাস্তবে network latency, server behavior, connection limits ইত্যাদির কারণে exact timing আলাদা হতে পারে।

21. Concurrency বনাম Parallelism

এই distinction খুব important।

Concurrency

একাধিক কাজ overlappingভাবে progress করছে।

Parallelism

একাধিক কাজ একই সময়ে সত্যিকার অর্থে আলাদা execution resource/core-এ চলছে।

JavaScript-এর সাধারণ async network code-এ `Promise.all()` ব্যবহার করলেই আমরা "parallel threads" তৈরি করছি—এমন বলা ঠিক নয়।

Better terminology:

Promise.all coordinates concurrent async operations.

22. Promise আগে Start করা বনাম Await আগে করা

ভুল performance pattern

```
const users = await getUsers();
const products = await getProducts();
const categories = await getCategories();
```

Independent হলে sequential।

Better

```
const usersPromise = getUsers();
const productsPromise = getProducts();
const categoriesPromise = getCategories();

const users = await usersPromise;
const products = await productsPromise;
const categories = await categoriesPromise;
```

আরও clean:

```
const [users, products, categories] = await Promise.all([
  getUsers(),
  getProducts(),
  getCategories()
]);
```

23. Promise.all()

সবচেয়ে গুরুত্বপূর্ণ Promise combinator-এর একটি।

```
const results = await Promise.all([
  promise1,
  promise2,
  promise3
]);
```

সব Promise fulfill হলে:

```
results
```

হবে array।

24. Promise.all() Example

```
const [users, products, orders] = await Promise.all([
  getUsers(),
  getProducts(),
  getOrders()
]);
```

এটা খুব common real-world pattern।

25. Promise.all() Result Ordering

একটি important বিষয়:

Promise যেই order-এ complete করবে, result সেই order-এ আসে না।

Input order preserve হয়।

```
const result = await Promise.all([
  slowPromise,
```

```
    fastPromise
  ]);
```

যদিও:

```
fastPromise → আগে complete
slowPromise → পরে complete
```

তবুও:

```
result[0] === slowPromiseResult
result[1] === fastPromiseResult
```

কারণ `Promise.all()` input order preserve করে।

26. `Promise.all()` Fail-Fast Behavior

ধরো:

```
const result = await Promise.all([
  promiseA,
  promiseB,
  promiseC
]);
```

যদি:

```
A → success
B → reject
C → success
```

তাহলে:

```
Promise.all()
```

reject করবে।

```
try {
  const result = await Promise.all([
    promiseA,
    promiseB,
    promiseC
  ]);
} catch (error) {
  console.error(error);
}
```

27. Important: `Promise.all()` Automatically Cancel করে না

এই বিষয়টি অনেকেই ভুল বোঝে।

```
await Promise.all([
  requestA(),
  requestB(),
  requestC()
]);
```

যদি `requestB()` reject করে, `Promise.all()` reject করবে।

কিন্তু `requestA()` বা `requestC()` already started হলে সেগুলো automatically cancel হয়ে যায় না।

Cancellation চাইলে আলাদা cancellation mechanism যেমন `AbortController` ব্যবহার করতে হয়।

এটা আমরা পরের advanced async doc-এ বিস্তারিত দেখব।

28. `Promise.allSettled()`

যখন তুমি চাও:

"সব operation শেষ হোক—success হোক বা failure হোক।"

তখন:

```
Promise.allSettled()
```

ব্যবহার করা হয়।

```
const results = await Promise.allSettled([
  promiseA,
  promiseB,
  promiseC
]);
```

29. Promise.allSettled() Result

Output structure:

```
[
  {
    status: "fulfilled",
    value: "A"
  },
  {
    status: "rejected",
    reason: Error
  },
  {
    status: "fulfilled",
    value: "C"
  }
]
```

30. Promise.all() বনাম Promise.allSettled()

Feature	Promise.all()	Promise.allSettled()
সব Promise wait করে?	না	হ্যাঁ
একটি reject করলে	immediately reject	সব শেষ হওয়া পর্যন্ত wait
সফল result	array	status/value
failure information	rejection	result-এর ভিতরে

Feature

Promise.all()

Promise.allSettled()

Use case

সবগুলো দরকার

প্রতিটির outcome দরকার

31. Promise.race()

Promise.race() প্রথম **settled** Promise-এর result দেয়।

Settled মানে:

fulfilled
অথবা
rejected

Example:

```
const result = await Promise.race([
  promiseA,
  promiseB
]);
```

32. Promise.race() Example

```
const first = new Promise((resolve) => {
  setTimeout(() => resolve("First"), 1000);
});

const second = new Promise((resolve) => {
  setTimeout(() => resolve("Second"), 2000);
});

const result = await Promise.race([
  first,
  second
]);

console.log(result);
```

Output:

First

33. Promise.race() Rejection

খুব important:

`race()` প্রথম **settled** Promise নেয়।

তাই প্রথমটা reject করলেও `race()` reject করবে।

```
const result = await Promise.race([
  Promise.reject(new Error("Failed")),
  someSlowPromise
]);
```

এখানে `race()` reject করবে।

34. Promise.any()

`Promise.any()` প্রথম **fulfilled** Promise-এর result দেয়।

```
const result = await Promise.any([
  promiseA,
  promiseB,
  promiseC
]);
```

যদি:

```
A → reject
B → reject
C → success
```

তাহলে:

C-এর result

পাওয়া যাবে।

35. `Promise.any()` বনাম `Promise.race()`

Situation	<code>race()</code>	<code>any()</code>
প্রথম fulfilled	return	return
প্রথম rejected	reject	ignore
প্রথম settled	return/reject	না
সব reject	reject	reject
সব reject হলে error	first rejection	<code>AggregateError</code>

36. `AggregateError`

যদি `Promise.any()` -এর সব Promise reject করে:

```
try {
  await Promise.any([
    Promise.reject(new Error("A")),
    Promise.reject(new Error("B")),
    Promise.reject(new Error("C"))
  ]);
} catch (error) {
  console.log(error instanceof AggregateError);
  console.log(error.errors);
}
```

Output conceptually:

```
true

[
  Error("A"),
  Error("B"),
  Error("C")
]
```

```
Error("C")
]
```

`AggregateError` multiple errors represent করতে ব্যবহৃত হয়।

37. চারটি **Promise Combinator** একসাথে

```
Promise.all()
  ↓
সব success দরকার

Promise.allSettled()
  ↓
সবগুলোর result দরকার

Promise.race()
  ↓
যে কোনো প্রথম settled result দরকার

Promise.any()
  ↓
যে কোনো প্রথম successful result দরকার
```

38. কোনটা কখন ব্যবহার করব?

সবগুলো **successful** হওয়া দরকার

```
await Promise.all([...]);
```

Success/failure যাই হোক, সবগুলোর **outcome** দরকার

```
await Promise.allSettled([...]);
```

যে **operation** আগে **settle** করবে সেটাই দরকার

```
await Promise.race([...]);
```

যে **operation** আগে **successfully complete** করবে সেটাই দরকার

```
await Promise.any([...]);
```

39. Async/Await in `for...of`

এটা খুব important।

```
for (const user of users) {  
  await saveUser(user);  
}
```

এখানে একটির কাজ শেষ হওয়ার পর পরেরটি শুরু হবে।

```
user 1  
↓  
save complete  
↓  
user 2  
↓  
save complete  
↓  
user 3
```

অর্থাৎ sequential।

40. `for...of` কেন Useful?

যখন প্রতিটি operation আগের operation-এর ওপর depend করে:

```
for (const item of items) {  
  await processItem(item);  
}
```

অথবা যখন intentionally rate/order maintain করতে চাও।

41. `map()` + `Promise.all()`

যদি items independent হয়:

```
const promises = users.map((user) => {  
  return saveUser(user);  
});  
  
await Promise.all(promises);
```

Short version:

```
await Promise.all(  
  users.map((user) => saveUser(user))  
);
```

এটা খুব common pattern ।

42. `forEach(async () => {})` Problem

অনেক developer প্রথমে এমন লেখে:

```
users.forEach(async (user) => {  
  await saveUser(user);  
});  
  
console.log("Done");
```

এখানে `"Done"` খুব সম্ভবত async operations শেষ হওয়ার আগেই print হবে।

কারণ `forEach()` async callback-এর returned Promise-এর জন্য wait করে না।

43. কেন `forEach` `await` করে না?

```
users.forEach(async (user) => {  
  await saveUser(user);  
});
```

`forEach()` callback invoke করে, কিন্তু callback-এর returned Promise collect/await করে না।

তাই:

```
forEach
  ↓
callback start
callback start
callback start
  ↓
forEach শেষ
  ↓
"Done"
  ↓
async কাজগুলো পরে complete
```

44. `forEach` এর বদলে কী ব্যবহার করব?

Sequential দরকার হলে

```
for (const user of users) {
  await saveUser(user);
}
```

Independent হলে

```
await Promise.all(
  users.map((user) => saveUser(user))
);
```

45. `map()` নিজে **Await** করে না

এটাও মনে রাখবে:

```
const result = users.map(async (user) => {
  return await saveUser(user);
});
```

`result` হবে:

```
[
  Promise,
  Promise,
  Promise
]
```

অর্থাৎ Promise-এর array।

তাই:

```
const result = await Promise.all(
  users.map(async (user) => {
    return saveUser(user);
  })
);
```

46. Sequential বনাম Concurrent Example

ধরো:

```
const tasks = [
  () => wait(2000),
  () => wait(3000),
  () => wait(1000)
];
```

Sequential

```
for (const task of tasks) {
  await task();
}
```

সময় প্রায়:

```
2 + 3 + 1 = 6 sec
```

Concurrent

```
await Promise.all(  
  tasks.map((task) => task())  
);
```

সময় roughly:

$\max(2, 3, 1) = 3 \text{ sec}$

47. Dependency Graph দিয়ে চিন্তা করো

ধরো:

```
Get User  
  ↓  
Get Orders  
  ↓  
Get Payment
```

এখানে sequential ।

কিন্তু:

```
      ↗ Get Users  
      |  
Start —+→ Get Products  
      |  
      ↘ Get Categories
```

এগুলো independent ।

তাই concurrent ।

Mental Rule

Dependency থাকলে **sequential**. **Dependency** না থাকলে **concurrency consider** করো.

48. Partial Dependency Example

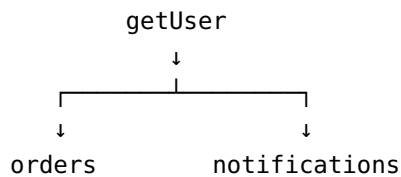
ধরো:

```
const user = await getUser();

const [orders, notifications] = await Promise.all([
  getOrders(user.id),
  getNotifications(user.id)
]);
```

এটা excellent pattern ।

কারণ:



`orders` আর `notifications` একে অপরের ওপর depend করে না।

49. `Promise.all()` Nested Dependency Pattern

খারাপ approach:

```
const user = await getUser();

const orders = await getOrders(user.id);

const notifications = await getNotifications(user.id);
```

যদি independent হয়:

```
const user = await getUser();

const [orders, notifications] = await Promise.all([
  getOrders(user.id),
```

```
    getNotifications(user.id)
  ]);
```

Performance better হতে পারে।

50. Async Function-এর Error Propagation

```
async function child() {
  throw new Error("Failed");
}

async function parent() {
  await child();
}

async function main() {
  try {
    await parent();
  } catch (error) {
    console.log(error.message);
  }
}
```

Error:

```
child
  ↓
parent
  ↓
main catch
```

অর্থাৎ async error Promise chain-এর মাধ্যমে propagate করে।

51. Error Rethrowing

কখনো error log করে আবার উপরে পাঠাতে হয়।

```
async function getUser() {
  try {
    return await fetchUser();
  } catch (error) {
    console.log(error);
    throw error;
  }
}
```

```
    } catch (error) {  
      console.error("User fetch failed");  
  
      throw error;  
    }  
  }  
}
```

এখানে:

```
catch  
↓  
log  
↓  
throw  
↓  
caller-এর কাছে error
```

52. Error Transform করা

```
async function getUser() {  
  try {  
    return await fetchUser();  
  } catch (error) {  
    throw new Error("Unable to load user", {  
      cause: error  
    });  
  }  
}
```

এতে original error-এর context রাখা যায়।

53. `async` Function-এর Return Type

```
async function test() {  
  return 10;  
}
```

Return type:

```
Promise<number>
```

TypeScript-এ:

```
async function test(): Promise<number> {  
  return 10;  
}
```

54. Async Function সবসময় Promise দেয়

```
async function getName() {  
  return "Shariar";  
}
```

এটা:

```
Promise<string>
```

এর মতো।

আর:

```
async function getUser() {  
  return {  
    id: 1  
  };  
}
```

হবে:

```
Promise<{ id: number }>
```

TypeScript ব্যবহার করলে এই concept খুব important।

55. Top-Level `await` — Intro

Modern JavaScript modules-এ কিছু environment-এ function-এর বাইরে সরাসরি:

```
const data = await fetchData();
```

লেখা যায়।

এটাকে বলা হয়:

Top-level await

Example:

```
const response = await fetch("/api/config");

const config = await response.json();

console.log(config);
```

এটা সাধারণ script context-এর বদলে supported ES module context-এ ব্যবহার করা হয়।

Top-level await আমরা 24.4-এ আরও বিস্তারিত দেখব।

56. Async/Await + Fetch

Real-world example:

```
async function getUsers() {
  const response = await fetch("/api/users");

  if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
  }

  const users = await response.json();

  return users;
}
```

তারপর:

```
async function main() {
  try {
    const users = await getUsers();

    console.log(users);
  } catch (error) {
    console.error(error);
  }
}
```

57. Multiple API Requests

```
async function loadDashboard() {
  const [profile, orders, notifications] = await Promise.all([
    getProfile(),
    getOrders(),
    getNotifications()
  ]);

  return {
    profile,
    orders,
    notifications
  };
}
```

এটা frontend/backend দুই জায়গাতেই খুব useful pattern ।

58. Sequential API Example

```
async function checkout() {
  const cart = await getCart();

  const total = calculateTotal(cart);

  const payment = await createPayment(total);

  const order = await createOrder(payment.id);
}
```

```
    return order;
}
```

এখানে dependency:

```
cart
↓
total
↓
payment
↓
order
```

তাই sequential হওয়া logical ।

59. Independent Operations একসাথে Start করা

আরেকটি useful pattern:

```
async function dashboard() {
  const profilePromise = getProfile();
  const ordersPromise = getOrders();
  const messagesPromise = getMessages();

  const profile = await profilePromise;
  const orders = await ordersPromise;
  const messages = await messagesPromise;

  return {
    profile,
    orders,
    messages
  };
}
```

এখানে তিনটি request শুরু হয়ে গেছে, তারপর results await করা হচ্ছে ।

তবে সাধারণত আরও clean:

```
async function dashboard() {
  const [profile, orders, messages] = await Promise.all([
```

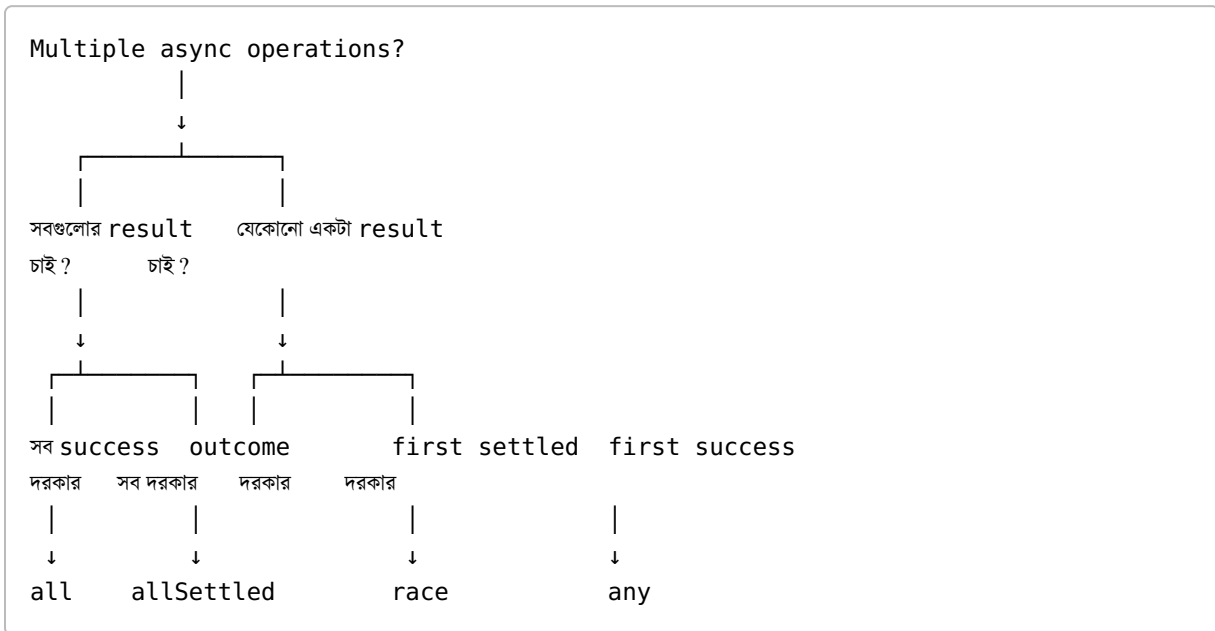
```

    getProfile(),
    getOrders(),
    getMessages()
  ]);

  return {
    profile,
    orders,
    messages
  };
}

```

60. Promise Combinator Decision Tree



61. Quick Comparison

Method	Success Condition	Failure Behavior	Main Use
<code>Promise.all()</code>	সব fulfill	প্রথম rejection-এ reject	সব data দরকার
<code>Promise.allSettled()</code>	সব settle	নিজে reject করে না	সব outcome দরকার
<code>Promise.race()</code>	প্রথম settle	প্রথম rejection-এ reject হতে পারে	first settled
<code>Promise.any()</code>	অন্তত ১ fulfill	সব reject হলে reject	first success

62. Common Mistake — Unnecessary Sequential Await

```
const a = await getA();  
const b = await getB();  
const c = await getC();
```

যদি:

A, B, C independent

হয়, তাহলে unnecessarily sequential ।

Better:

```
const [a, b, c] = await Promise.all([  
  getA(),  
  getB(),  
  getC()  
]);
```

63. Common Mistake — `forEach(async...)`

Avoid:

```
items.forEach(async (item) => {  
  await process(item);  
});
```

Use:

```
for (const item of items) {  
  await process(item);  
}
```

অথবা:

```
await Promise.all(
  items.map((item) => process(item))
);
```

64. Common Mistake — `await` inside `map()` ভুলভাবে ব্যবহার

এটা valid:

```
const results = await Promise.all(
  items.map(async (item) => {
    return await process(item);
  })
);
```

কিন্তু অনেক সময় inner `await` প্রয়োজন নেই:

```
const results = await Promise.all(
  items.map((item) => process(item))
);
```

65. Common Mistake — `Promise.all()` দিয়ে Dependency ভাঙা

এটা করা যাবে না যদি:

```
const user = await getUser();

const orders = await getOrders(user.id);
```

কারণ `orders` -এর জন্য `user.id` দরকার।

তাই:

```
const [user, orders] = await Promise.all([
  getUser(),
  getOrders(user.id)
]);
```

এটা invalid logic, কারণ `user` তখনো পাওয়া যায়নি।

66. Correct Partial Concurrency

```
const user = await getUser();

const [orders, notifications, messages] = await Promise.all([
  getOrders(user.id),
  getNotifications(user.id),
  getMessages(user.id)
]);
```

এটাই real-world async optimization-এর একটি important pattern।

67. Async/Await Mental Model

এই code:

```
async function main() {
  console.log("A");

  const result = await getData();

  console.log("B");

  return result;
}
```

মনে করবে:

```
main starts
↓
"A"
↓
```

```
getData() starts
↓
await
↓
main pauses
↓
other JS work continues
↓
Promise settles
↓
main resumes
↓
"B"
↓
return result
↓
Promise fulfilled
```

68. Sequential বনাম Concurrent — সবচেয়ে Important Concept

Sequential

```
const a = await getA();
const b = await getB();
const c = await getC();
```

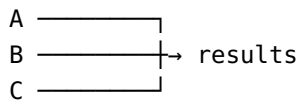
Graph:

A → B → C

Concurrent

```
const [a, b, c] = await Promise.all([
  getA(),
  getB(),
  getC()
]);
```

Graph:



69. কখন **Sequential** ব্যবহার করব?

Use sequential যখন:

- পরের operation আগের result-এর ওপর depend করে
- order গুরুত্বপূর্ণ
- rate limit maintain করতে চাও
- previous operation সফল না হলে পরেরটি চালানো উচিত নয়
- state transition একটার পর একটা হতে হবে

Example:

```
const account = await createAccount();

const profile = await createProfile(account.id);

await sendWelcomeEmail(profile.email);
```

70. কখন **Concurrent** ব্যবহার করব?

Use concurrency যখন:

- operations independent
- সব result eventually দরকার
- performance improve করতে চাও
- network/API calls একে অপরের ওপর depend করে না

Example:

```
const [users, products, categories] = await Promise.all([
  getUsers(),
  getProducts(),
  getCategories()
]);
```

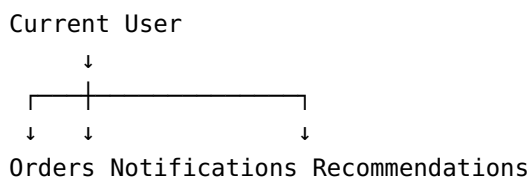
71. Real-World Dashboard Example

```
async function loadDashboard() {
  const user = await getCurrentUser();

  const [
    orders,
    notifications,
    recommendations
  ] = await Promise.all([
    getOrders(user.id),
    getNotifications(user.id),
    getRecommendations(user.id)
  ]);

  return {
    user,
    orders,
    notifications,
    recommendations
  };
}
```

এখানে:



এটাই **dependency-aware concurrency**।

72. Practice — Level 1

Question 1

Output কী হবে?

```
async function test() {
  return 10;
}
```

```
}  
  
console.log(test());
```

Question 2

এখানে output কী হবে?

```
async function test() {  
  const value = await Promise.resolve(20);  
  
  console.log(value);  
}  
  
test();
```

Question 3

কোনটি sequential?

```
const a = await getA();  
const b = await getB();
```

নাকি:

```
const [a, b] = await Promise.all([  
  getA(),  
  getB()  
]);
```

73. Practice — Level 2

Question 4

এই code-এর problem কী?

```
users.forEach(async (user) => {  
  await saveUser(user);  
});  
  
console.log("All users saved");
```

এটা ঠিক করো।

Question 5

যদি:

```
A = 2 sec  
B = 3 sec  
C = 4 sec
```

হয়, তাহলে:

```
await A;  
await B;  
await C;
```

roughly কত সময় লাগতে পারে?

আর:

```
await Promise.all([A, B, C]);
```

roughly কত সময় লাগতে পারে?

74. Practice — Level 3

ধরো:

```
const user = await getUser();  
  
const orders = await getOrders(user.id);
```

```
const notifications = await getNotifications(user.id);  
  
const messages = await getMessages(user.id);
```

এখানে `orders`, `notifications`, `messages` একে অপরের ওপর `depend` করে না।

এটাকে `optimize` করো।

Expected pattern:

```
const user = await getUser();  
  
const [orders, notifications, messages] = await Promise.all([  
  getOrders(user.id),  
  getNotifications(user.id),  
  getMessages(user.id)  
]);
```

75. Practice — Level 4

নিচের কোন combinator ব্যবহার করবে?

A

সবগুলো `successful` হওয়া `mandatory`।

```
API A  
API B  
API C
```

B

সব API-এর `success/failure status` দরকার।

C

যে API আগে `response` দেবে সেটাই দরকার—even if rejected।

D

যে API আগে `successfully response` দেবে সেটাই দরকার।

76. Tricky Output Question

```
async function test() {  
  console.log("A");  
  
  await Promise.resolve();  
  
  console.log("B");  
}  
  
console.log("C");  
  
test();  
  
console.log("D");
```

নিজে output predict করো।

Hint:

`await` পুরো JavaScript thread block করে না।

77. Real-World Challenge

একটি e-commerce product page-এ:

```
Product  
Reviews  
Recommendations  
Related Products
```

চারটি API আছে।

`Product` API-এর result ছাড়া `Reviews` API call করা যায় না।

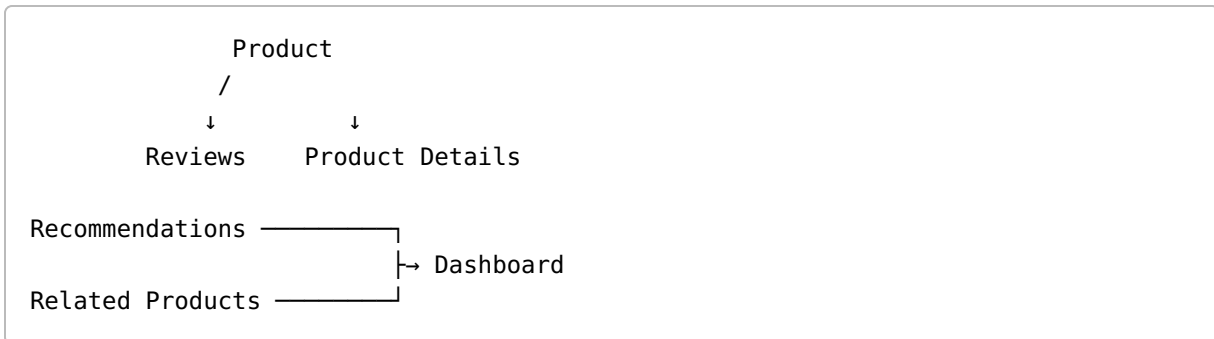
কিছু:

Recommendations
Related Products

এর জন্য Product API দরকার নেই।

Optimal async structure লিখো।

একটি সম্ভাব্য dependency graph:



এখানে কোন operations sequential এবং কোনগুলো concurrent হবে সেটা নিজে determine করো।

78. Interview Questions

Basic

1. `async` function কী return করে?
2. `await` কী করে?
3. `await` কি JavaScript thread block করে?
4. `async/await` এবং Promise chaining-এর relation কী?
5. `async function` -এ `throw` করলে কী হয়?

Intermediate

1. `Promise.all()` কীভাবে কাজ করে?
2. `Promise.all()` result order কীভাবে determine হয়?
3. `Promise.all()` এবং `Promise.allSettled()` এর পার্থক্য কী?
4. `Promise.race()` এবং `Promise.any()` এর পার্থক্য কী?
5. `AggregateError` কখন পাওয়া যায়?
6. `forEach(async () => {})` কেন problematic?
7. `map()` + `Promise.all()` কেন common pattern?
8. Sequential আর concurrent execution-এর পার্থক্য কী?
9. কখন sequential execution prefer করবে?
10. কখন concurrency prefer করবে?

Advanced

1. `Promise.all()` reject করলে already-started operations কি automatically cancel হয়?
 2. `await` internally Promise continuation-এর সাথে কীভাবে কাজ করে?
 3. Dependency-aware concurrency কী?
 4. `try/catch` কেন un-awaited floating Promise-এর error ধরতে পারে না?
 5. Network I/O-এর ক্ষেত্রে concurrency আর parallelism-এর মধ্যে পার্থক্য কী?
-

79. Quick Revision

```
async
↓
always returns Promise
```

```
await promise
↓
fulfilled → value
rejected → throw error
```

```
Sequential
A → B → C
```

```
Concurrent
A ┌─┐
B ┌─┼─┐ Promise.all()
C ┌─┐
```

```
Promise.all()
→ all success দরকার
```

```
Promise.allSettled()
→ সব outcome দরকার
```

```
Promise.race()
→ first settled
```

```
Promise.any()  
→ first fulfilled
```

```
for...of + await  
→ sequential
```

```
map() + Promise.all()  
→ concurrent async operations
```

```
forEach(async ...)  
→ সাধারণত await করার জন্য ব্যবহার করবে না
```

80. Mastery Checklist

এই document শেষ করার পর তুমি explain করতে পারবে:

- `[]` `async` function কী
- `[]` কেন `async` function সবসময় `Promise` return করে
- `[]` `await` কীভাবে কাজ করে
- `[]` `await` কেন পুরো `JavaScript` block করে না
- `[]` `await` normal value handle করতে পারে
- `[]` `thenable` কী
- `[]` `async` function-এ `return` কীভাবে `Promise` তৈরি করে
- `[]` `async` function-এ `throw` কীভাবে `rejection` তৈরি করে
- `[]` `try/catch/finally` with `async/await`
- `[]` error propagation
- `[]` sequential execution
- `[]` concurrent execution
- `[]` concurrency বনাম parallelism
- `[]` `Promise.all()`
- `[]` `Promise.allSettled()`
- `[]` `Promise.race()`
- `[]` `Promise.any()`
- `[]` `AggregateError`
- `[]` `Promise` result ordering
- `[]` fail-fast behavior
- `[]` `for...of` + `await`
- `[]` `map()` + `Promise.all()`
- `[]` কেন `forEach(async...)` problematic
- `[]` dependency-aware concurrency

- [] কখন sequential আর কখন concurrent code লিখতে হবে

Final Mental Model

Async JavaScript-এ সবচেয়ে important প্রশ্ন হলো:

"এই কাজটা কি আগের কাজের **result**-এর ওপর **depend** করে?"

যদি হ্যাঁ:

```
const a = await getA();
const b = await getB(a);
const c = await getC(b);
```

➡ Sequential.

যদি না:

```
const [a, b, c] = await Promise.all([
  getA(),
  getB(),
  getC()
]);
```

➡ Concurrent.

আর Promise combinators মনে রাখবে:

```
all
↓
সবগুলো সফল চাই

allSettled
↓
সবগুলোর outcome চাই

race
↓
first settled চাই

any
```

↓

first successful চাই

এটাই modern JavaScript async programming-এর অন্যতম core mental model ।